Lesson 12 Teacher's Script

Hearing It Happen

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week you hit Level 2 Complete - a real, replayable game with multiple waves. Today's a lighter lesson, and honestly, one of the most fun ones all semester: your game is about to start sounding like a game."

"From here through Lesson 15, we're not adding new mechanics anymore. We're making the mechanics you already have feel good - sound this week, animation next week, difficulty scaling after that, and a final polish pass before the capstone."

[0:05-0:15] The Sound Payoff: Four Sounds, Four Existing Moments

"Quick question - by a show of hands, who's built one of the optional 'add a sound' ideas along the way? Lesson 2's movement beep, Lesson 3's shoot sound, Lesson 4's hit sound, Lesson 6's victory sound?" [Pause, note roughly how many hands go up]

"However many hands went up, here's the deal: today, four sounds become required for everyone - a shot, an explosion, a wave-clear cue, and a game-over cue. If you already built some of these, you're mostly just verifying and adding two brand-new ones. If you didn't build any, today's Handout gives you everything from scratch - it's written to work either way."

"Here's the key idea: every one of these four sounds gets added to a spot in your code that ALREADY exists - the exact moment you fire a shot, the exact moment a bullet hits an enemy, the exact branch where a wave advances, the exact branch where you lose. No new logic. Just one more line at each of four places you already built."

[Timing checkpoint: wrap by 0:15]

[0:15-0:35] One Shared Sound for Three Outcomes

[Demo cue: switch to GameMaker, open a project with working multi-wave logic from Lesson 11, GML Visual tab open on obj_bullet's Collision event]

"Shoot and hit sounds are easy - they already live on obj_player and obj_bullet, one sound each, and if you built them back in Lessons 3 and 4, they need zero changes today. The more interesting part is the three sounds that can happen when a wave ends: a win, a wave-clear, or a loss. Only one of those three can ever be true at once - so I don't need any shared node or extra bookkeeping to keep them from overlapping. I just drop an Audio Play Sound block directly into each of the three branches that already decide which outcome happened."

"Watch: right on Lesson 11's If Variable block - the one checking whether this was the last wave - I add an Audio Play Sound block (sfx_win) on the True output, and a second Audio Play Sound block (sfx_wave) on the False output. Then, on obj_enemy's escape branch, one more Audio Play Sound block (sfx_lose) inside the lives-reaches-zero check." [Build the three blocks live, demonstrate a wave-clear sound followed by a win sound on the final wave]

"This is the same reuse idea you've seen all semester, just applied to a sound block instead of a function - and it's a nice example of GameMaker's block-for-block honesty: three blocks, three branches, no shared object standing in the middle of it. Godot and Unity students do the same idea slightly differently, routing all three clips through one shared audio player node instead - different mechanism, same underlying idea of one outcome, one sound, no overlap."

[Timing checkpoint: wrap by 0:35]

[0:35-0:45] Why None of Today's Sounds Need a Retrigger Guard

"Here's something worth noticing: none of today's four sounds need any kind of 'don't replay this every frame' guard. Does anyone remember why Lesson 2's beep-beep sound needed an explicit stop call, but a one-shot sound like today's shoot or hit sound doesn't?" [Pause for answers] "Right - because a one-shot sound fires once and finishes on its own, it doesn't need turning off. But there's a second reason today's four sounds are safe: every single one of them lives inside a branch that ONLY runs once per actual event - a real hit, a real wave transition, a real loss. Nothing here is being checked every single frame."

"TIC-80 students specifically: if you built Lesson 6's optional win sound, you may have needed a guard flag back then, because the check ran every frame in a continuous loop. As of Lesson 10 and 11's restructuring, that's no longer true - the win-check now lives inside a one-time hit branch. If you've still got that old guard flag lying around, it's safe to remove it today."

[Timing checkpoint: wrap by 0:45]

[0:45-0:55] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one. Two new sound files are provided (sfx_wave.wav, sfx_lose.wav); everything else you may already have from earlier optional ideas."

[Hand out engine-specific handouts]

"Goal for today: play a full round out loud - a shot, a hit, an early wave clear, and separately, a loss - and confirm all four sound genuinely different from each other. If you already had a win sound, make sure it still plays correctly and doesn't overlap the wave-clear sound."

"Questions before we start lab? ... Let's make some noise."

Lab Time Notes

- The most common issue this week isn't a bug so much as a missing step - remind students who skipped the optional Lessons 2/3/4/6 ideas that today's Handout gives them full instructions, not just 'add two sounds.'
- Watch for a sound call placed outside the correct if/else branch, causing two round-ending sounds to play together (e.g. wave-clear and win firing on the same frame).
- This is a good week to actually listen as you circulate, not just watch the screen - a wrong or missing sound is often faster to catch by ear than by reading code.

- If a student's speakers/headphones aren't working, sound logic can still be verified visually for now (e.g. a temporary print/debug line) - don't let a hardware issue block progress on the actual lesson goal.
- This is a lighter week - if a cluster finishes early, point them at the Level 3 ideas (pitch variation, the optional win sound if not yet built) rather than moving ahead to next week's material.